TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐẠI HỌC QUỐC GIA TP.HCM



BÀI THỰC HÀNH VI XỬ LÝ - VI ĐIỀU KHIỂN

LAB 3: BUTTONS / SWITCHES

NGUYỄN ĐỖ KHÁNH TRÌNH - 2353237

30/10/2026

Mục lục

Nội Dung Thực Hành **3**

1 Exercise 1: FSM sketch **3**

2 Exercise 2: Proteus Schematic **4**

3 Exercise 3: STM32 Project with 10 ms interrupt **5**

4 Exercise 4: Modify Timer Parameters **5**

5 Exercise 5: Button debouncing **6**

5.1 Ý tưởng 6

6 Exercise 6: Displaying modes **8**

6.1 LED 7 đoạn 8

6.2 Nháy đèn theo mode 8

7 Exercise 7: Increasing time duration for the red LEDs **10**

8 Exercise 8: Increasing time duration for the amber LEDs **10**

9 Exercise 9: Increasing time duration for the green LEDs **10**

10 Exercise 10: Final project **10**

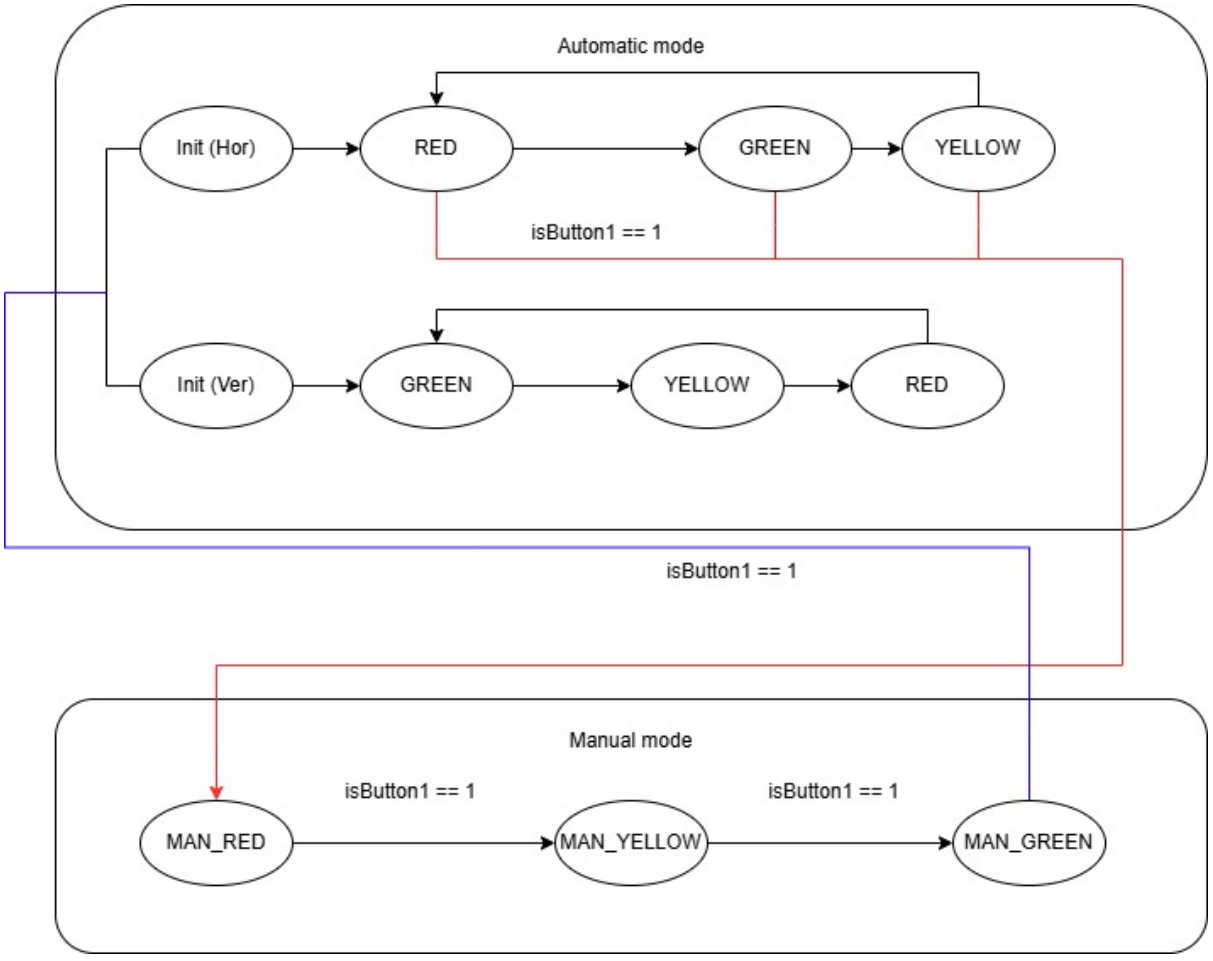
Nội Dung Thực Hành

Code và Schematic có thể được xem trực tiếp và tải về tại GitHub Repository: Microcontroller Reports
Demo Lab 3 bằng Video: Link Youtube

Đặc tả hệ thống

- 12 LED giao thông (4 đỏ, 4 vàng, 4 xanh) cho 2 hướng: ngang (RED, YEL, GRN – PA7..PA9) và dọc (RED1, YEL1, GRN1 – PA10..PA12). Hai đèn cùng hướng nối chung một chân.
- 4 LED 7 đoạn anode chung, chia thành 2 cặp: cặp A (EN0, EN1 – PA13, PA14; đoạn PB0..PB6) và cặp B (EN2, EN3 – PB14, PB15; đoạn PB7..PB13). Mỗi chân chung được cấp nguồn qua một transistor PNP.
- 3 nút nhấn B1, B2, B3 (PA0..PA2) nối xuống GND, có điện trở kéo lên 10 kΩ: nhấn đọc 0.
- Mode 1: chạy bình thường. Mode 2, 3, 4: chỉnh thời gian đèn đỏ, vàng, xanh. B1 đổi mode, B2 tăng giá trị (1–99), B3 lưu giá trị.

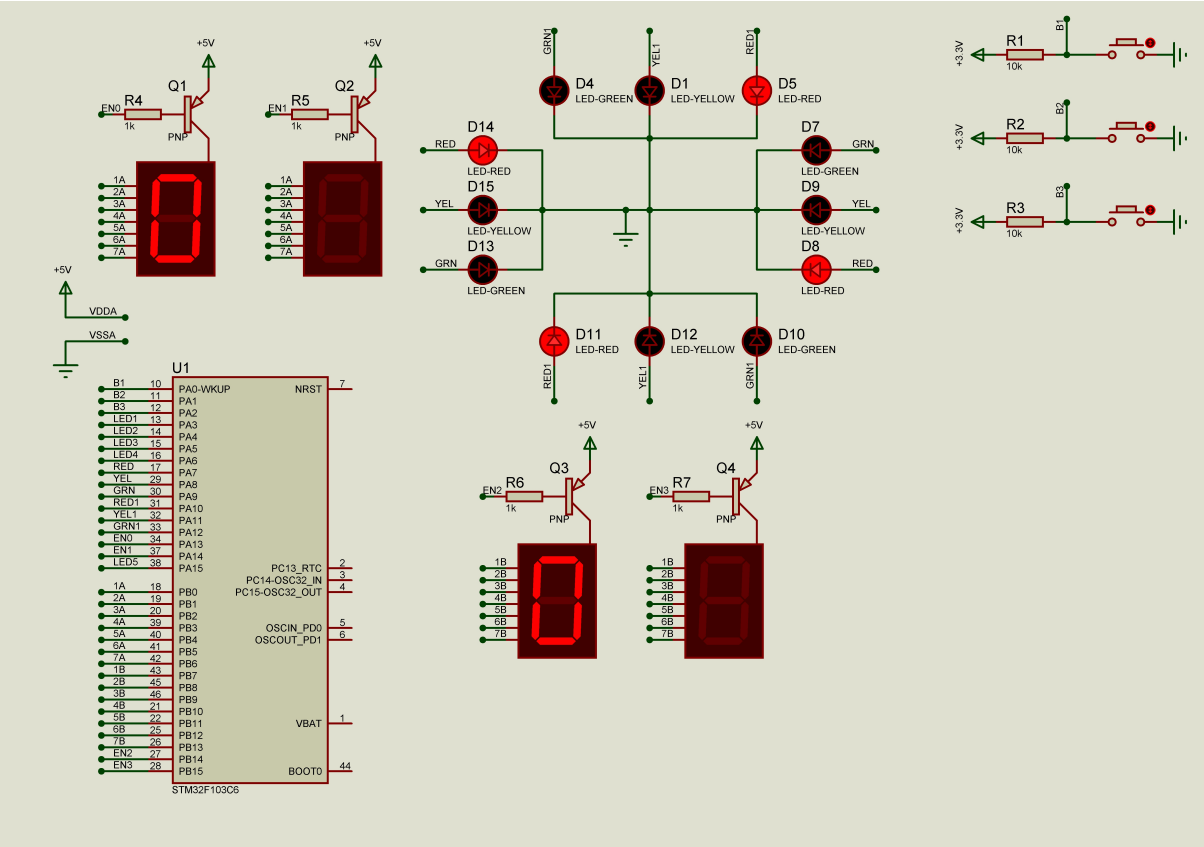
1 Exercise 1: FSM sketch



Hệ thống gồm hai nhóm trạng thái:

- **Automatic mode (Mode 1):** hai FSM chạy song song. Hướng ngang bắt đầu từ RED, hướng dọc bắt đầu từ GREEN; mỗi hướng đi theo vòng RED → GREEN → YELLOW → RED. Để hai hướng không bao giờ cùng xanh, luôn giữ $T_{RED} = T_{GREEN} + T_{YELLOW}$ (mặc định $5 = 3 + 2$ giây).
- **Manual mode:** MAN_RED (Mode 2) → MAN_YELLOW (Mode 3) → MAN_GREEN (Mode 4). Ở bất kỳ trạng thái nào của Automatic mode, nhấn B1 sẽ sang MAN_RED; ở MAN_GREEN nhấn B1 thì quay về Init của Automatic mode.

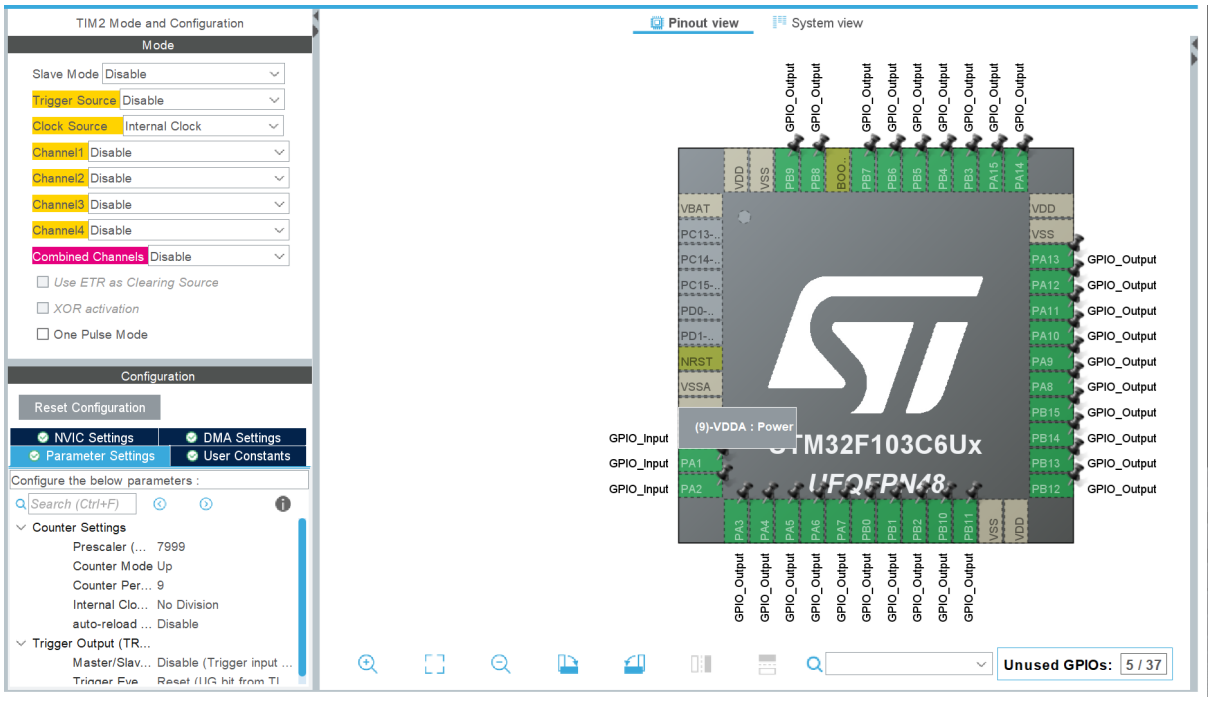
2 Exercise 2: Proteus Schematic



Chân	Label	Chức năng
PA0, PA1, PA2	B1, B2, B3	Nút nhấn (input, pull-up ngoài 10 kΩ)
PA7, PA8, PA9	RED, YEL, GRN	Đèn hướng ngang (SET = sáng)
PA10, PA11, PA12	RED1, YEL1, GRN1	Đèn hướng dọc (SET = sáng)
PA13, PA14	EN0, EN1	Chọn LED 7 đoạn cặp A (0 = sáng)
PB0 – PB6	SA0 – SA6	Đoạn a–g của cặp A (0 = sáng)
PB7 – PB13	SB0 – SB6	Đoạn a–g của cặp B (0 = sáng)
PB14, PB15	EN2, EN3	Chọn LED 7 đoạn cặp B (0 = sáng)

Emitter của các transistor PNP và chân VDDA nối +3.3 V (cùng mức với chân MCU), để khi EN = 1 thì PNP tắt hẳn.

3 Exercise 3: STM32 Project with 10 ms interrupt



Cấu hình trong STM32CubeMX:

- Clock: HSI 8 MHz, không dùng PLL.
- SYS → Debug: *No Debug*, để dùng PA13, PA14 làm GPIO.
- TIM2: Clock Source = Internal Clock, Prescaler = 7999, Counter Period = 9, bật *TIM2 global interrupt* trong NVIC.

Chu kỳ ngắt:

$$T = \frac{(PSC+1)(ARR+1)}{f_{CLK}} = \frac{8000 \times 10}{8\,000\,000} = 10 \text{ ms}$$

4 Exercise 4: Modify Timer Parameters

Yêu cầu: khi đổi chu kỳ ngắt (1 ms hoặc 100 ms), hệ thống vẫn chạy đúng (đèn vẫn nháy 2 Hz).

Chu kỳ ngắt	Prescaler	Counter Period	TIMER_CYCLE
1 ms	7999	0	1
1 ms	799	9	1
10 ms	7999	9	10
100 ms	7999	99	100

Lưu ý: không thể dùng Prescaler = 79999 vì thanh ghi 16 bit chỉ chứa tối đa 65535. Giải pháp:

- Mọi thời gian trong chương trình đều viết bằng mili-giây (`setTimer(TMR_BLINK, 250)`, `setTimer(TMR_SECOND, 1000)`...), không viết bằng số lần ngắt.
- Chỉ có **một** hằng `TIMER_CYCLE` mô tả chu kỳ ngắt. `setTimer()` tự đổi ms sang số lần ngắt: `ticks = duration / TIMER_CYCLE`; nếu kết quả bằng 0 thì lấy 1 để timer vẫn chạy.
- Thời gian nhấn giữ 1 giây cũng tính theo `TIMER_CYCLE`: `LONG_PRESS_TIME = 1000 / TIMER_CYCLE`.

Vì vậy khi đổi chu kỳ ngắt chỉ cần sửa Prescaler/Counter Period trong CubeMX và sửa dòng `#define TIMER_CYCLE`.

Listing 1: Software timer

```
#define TIMER_CYCLE  10

#define NUM_TIMERS   4
#define TMR_SECOND   0   /* dem giay o mode 1        */
#define TMR_SCAN     1   /* quet LED 7 doan          */
#define TMR_BLINK    2   /* nhay den 2 Hz o mode 2-4 */
#define TMR_REPEAT   3   /* tu tang khi giu nut 2    */

volatile int timer_counter[NUM_TIMERS];
volatile int timer_flag[NUM_TIMERS];

void setTimer(int id, int duration) {        /* duration tinh bang ms */
    int ticks = duration / TIMER_CYCLE;
    if (ticks < 1) ticks = 1;
    timer_counter[id] = ticks;
    timer_flag[id] = 0;
}

int isTimerExpired(int id) {
    return timer_flag[id];
}

void timerRun(void) {
    for (int i = 0; i < NUM_TIMERS; i++) {
        if (timer_counter[i] > 0) {
            timer_counter[i]--;
            if (timer_counter[i] == 0) timer_flag[i] = 1;
        }
    }
}
```

5 Exercise 5: Button debouncing

5.1 Ý tưởng

- Hàm `button_reading()` được gọi trong ngắt TIM2, mỗi 10 ms đọc 3 nút một lần.
- Lưu 3 lần đọc gần nhất (`keyReg0..2`); chỉ khi cả 3 giống nhau (ổn định 20–30 ms, lâu hơn thời gian dội khoảng 10 ms) mới chấp nhận trạng thái mới.

- Khi trạng thái ổn định chuyển từ thả sang nhấn, bật cờ `pressFlag`. Hàm `isButtonPressed()` trả về 1 đúng một lần cho mỗi lần nhấn rồi xóa cờ, nên giữ nút không bị tính thành nhiều lần nhấn.
- Nếu nút được giữ liên tục qua 1 giây, bật cờ `longFlag` để dùng cho chức năng tự tăng giá trị.
- Nhấn B1 chuyển mode: Mode 1 → 2 → 3 → 4 → 1 (xem Exercise 10).

Listing 2: Đọc nút nhấn và chống dội phím

```
#define NUM_BUTTONS      3
#define BTN_MODE         0   /* B1 - PA0: doi mode     */
#define BTN_INC          1   /* B2 - PA1: tang gia tri */
#define BTN_SET          2   /* B3 - PA2: luu gia tri  */

#define PRESSED          GPIO_PIN_RESET
#define RELEASED         GPIO_PIN_SET
#define LONG_PRESS_TIME  (1000 / TIMER_CYCLE)   /* 1 giay */

GPIO_TypeDef *btnPort[NUM_BUTTONS] = { B1_GPIO_Port, B2_GPIO_Port, B3_GPIO_Port };
uint16_t      btnPin[NUM_BUTTONS]  = { B1_Pin,       B2_Pin,       B3_Pin };

GPIO_PinState keyReg0[NUM_BUTTONS]     = { RELEASED, RELEASED, RELEASED };
GPIO_PinState keyReg1[NUM_BUTTONS]     = { RELEASED, RELEASED, RELEASED };
GPIO_PinState keyReg2[NUM_BUTTONS]     = { RELEASED, RELEASED, RELEASED };
GPIO_PinState stableState[NUM_BUTTONS] = { RELEASED, RELEASED, RELEASED };

int          pressCounter[NUM_BUTTONS];
volatile int pressFlag[NUM_BUTTONS];
volatile int longFlag[NUM_BUTTONS];

void button_reading(void) {                  /* goi trong ISR moi 10 ms */
    for (int i = 0; i < NUM_BUTTONS; i++) {
        keyReg2[i] = keyReg1[i];
        keyReg1[i] = keyReg0[i];
        keyReg0[i] = HAL_GPIO_ReadPin(btnPort[i], btnPin[i]);

        /* chi tin khi 3 lan doc lien tiep giong nhau */
        if (keyReg0[i] == keyReg1[i] && keyReg1[i] == keyReg2[i]) {
            if (keyReg2[i] != stableState[i]) {          /* vua doi trang thai */
                stableState[i] = keyReg2[i];
                if (stableState[i] == PRESSED) {
                    pressFlag[i] = 1;
                    pressCounter[i] = LONG_PRESS_TIME;
                } else {
                    longFlag[i] = 0;
                }
            } else if (stableState[i] == PRESSED && pressCounter[i] > 0) {
                pressCounter[i]--;                       /* dang giu nut */
                if (pressCounter[i] == 0) longFlag[i] = 1;
            }
        }
    }
}

int isButtonPressed(int index) {             /* tra 1 dung MOT lan moi lan nhan */
```

```
    if (pressFlag[index] == 1) {
        pressFlag[index] = 0;
        return 1;
    }
    return 0;
}

int isButtonLongPressed(int index) {         /* = 1 khi dang giu qua 1 giay */
    return longFlag[index];
}
```

Listing 3: Ngắt TIM2: chỉ đọc nút và đếm timer

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    button_reading();
    timerRun();
  }
}
```

6 Exercise 6: Displaying modes

6.1 LED 7 đoạn

- Hai cặp LED 7 đoạn dùng hai bus đoạn riêng nên quét song song: lượt 0 bật EN0 và EN2 (hàng chục), lượt 1 bật EN1 và EN3 (hàng đơn vị).
- Mỗi lượt 10 ms, mỗi cặp 2 LED $\Rightarrow$ tần số quét $f = \frac{1}{2\times 10\text{ ms}} = 50\,\text{Hz}$, mắt không thấy nháy.
- Thứ tự mỗi lượt: tắt hết EN $\rightarrow$ xuất mã đoạn mới $\rightarrow$ bật EN của LED cần sáng, tránh bóng mờ của số cũ.
- Mode 1: cặp A hiện thời gian còn lại của hướng ngang, cặp B hiện của hướng dọc.
- Mode 2–4: cặp A hiện giá trị đang chỉnh, cặp B hiện số mode (02, 03, 04).

6.2 Nháy đèn theo mode

Ở Mode 2, 3, 4, tất cả đèn của màu đang chỉnh (ở cả hai hướng) đảo trạng thái mỗi 250 ms, tức nháy với chu kỳ 500 ms, tần số 2 Hz. Các đèn còn lại tắt.

Listing 4: Hiển thị LED 7 đoạn và đèn giao thông

```
#define LIGHT_OFF     0
#define LIGHT_RED     1
#define LIGHT_GREEN   2
#define LIGHT_YELLOW  3

/* 1 = doan sang. Thu tu cot: a b c d e f g */
const int digit[10][7] = {
    {1,1,1,1,1,1,0},  /* 0 */
    {0,1,1,0,0,0,0},  /* 1 */
```

```
    {1,1,0,1,1,0,1},  /* 2 */
    {1,1,1,1,0,0,1},  /* 3 */
    {0,1,1,0,0,1,1},  /* 4 */
    {1,0,1,1,0,1,1},  /* 5 */
    {1,0,1,1,1,1,1},  /* 6 */
    {1,1,1,0,0,0,0},  /* 7 */
    {1,1,1,1,1,1,1},  /* 8 */
    {1,1,1,1,0,1,1}   /* 9 */
};

const uint16_t segA[7] = { SA0_Pin, SA1_Pin, SA2_Pin, SA3_Pin, SA4_Pin, SA5_Pin,
    SA6_Pin };
const uint16_t segB[7] = { SB0_Pin, SB1_Pin, SB2_Pin, SB3_Pin, SB4_Pin, SB5_Pin,
    SB6_Pin };

int bufA[2] = {0, 0};      /* cap A: EN0, EN1 */
int bufB[2] = {0, 0};      /* cap B: EN2, EN3 */
int scanIndex = 0;

void writeDigit(const uint16_t seg[7], int num) {      /* RESET = doan sang */
    for (int i = 0; i < 7; i++) {
        HAL_GPIO_WritePin(GPIOB, seg[i], digit[num][i] ? GPIO_PIN_RESET : GPIO_PIN_SET
    );
    }
}

void setBufferA(int num) { bufA[0] = num / 10; bufA[1] = num % 10; }
void setBufferB(int num) { bufB[0] = num / 10; bufB[1] = num % 10; }

/* Hai cap quet song song. PNP: EN = 0 la sang */
void scan7SEG(void) {
    HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin, GPIO_PIN_SET);   /* 1. tat het */
    HAL_GPIO_WritePin(GPIOB, EN2_Pin | EN3_Pin, GPIO_PIN_SET);

    writeDigit(segA, bufA[scanIndex]);                           /* 2. xuat so */
    writeDigit(segB, bufB[scanIndex]);

    if (scanIndex == 0) {                                        /* 3. bat LED */
        HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET);
    } else {
        HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET);
    }
    scanIndex = 1 - scanIndex;
}

/* Den giao thong: cathode xuong GND -> SET = sang */
void setLightHor(int color) {
    HAL_GPIO_WritePin(RED_GPIO_Port, RED_Pin, (color == LIGHT_RED)    ? GPIO_PIN_SET :
     GPIO_PIN_RESET);
    HAL_GPIO_WritePin(YEL_GPIO_Port, YEL_Pin, (color == LIGHT_YELLOW) ? GPIO_PIN_SET :
     GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GRN_GPIO_Port, GRN_Pin, (color == LIGHT_GREEN)  ? GPIO_PIN_SET :
     GPIO_PIN_RESET);
}

void setLightVer(int color) {
```

```
62      HAL_GPIO_WritePin(RED1_GPIO_Port, RED1_Pin, (color == LIGHT_RED)    ? GPIO_PIN_SET
        : GPIO_PIN_RESET);
63      HAL_GPIO_WritePin(YEL1_GPIO_Port, YEL1_Pin, (color == LIGHT_YELLOW) ? GPIO_PIN_SET
        : GPIO_PIN_RESET);
64      HAL_GPIO_WritePin(GRN1_GPIO_Port, GRN1_Pin, (color == LIGHT_GREEN)  ? GPIO_PIN_SET
        : GPIO_PIN_RESET);
65  }
66
67  void toggleColor(int color) {               /* nhay 1 mau o ca hai huong */
68      if (color == LIGHT_RED)    HAL_GPIO_TogglePin(GPIOA, RED_Pin | RED1_Pin);
69      if (color == LIGHT_YELLOW) HAL_GPIO_TogglePin(GPIOA, YEL_Pin | YEL1_Pin);
70      if (color == LIGHT_GREEN)  HAL_GPIO_TogglePin(GPIOA, GRN_Pin | GRN1_Pin);
71  }
```

7 Exercise 7: Increasing time duration for the red LEDs

Ở Mode 2 (trạng thái MAN_RED):

- Khi vào mode, `tempValue` được nạp bằng thời gian đèn đỏ hiện tại.
- Nhấn B2: `tempValue` tăng 1, vượt 99 thì quay về 1. Giữ B2 quá 1 giây thì tự tăng mỗi 200 ms.
- Nhấn B3: lưu `tempValue` vào `durRed`. Để giữ $T_{RED} = T_{GREEN} + T_{YELLOW}$, chương trình tự tính lại $T_{GREEN} = T_{RED} - T_{YELLOW}$ (tối thiểu 1 giây).
- Nếu chuyển mode mà chưa nhấn B3 thì giá trị đang chỉnh bị bỏ.

8 Exercise 8: Increasing time duration for the amber LEDs

Ở Mode 3 (MAN_YELLOW), B2 và B3 hoạt động giống Mode 2 nhưng áp dụng cho đèn vàng. Khi lưu, chương trình tính lại $T_{RED} = T_{GREEN} + T_{YELLOW}$ (tối đa 99 giây).

9 Exercise 9: Increasing time duration for the green LEDs

Ở Mode 4 (MAN_GREEN), B2 và B3 hoạt động giống Mode 2 nhưng áp dụng cho đèn xanh. Khi lưu, chương trình cũng tính lại $T_{RED} = T_{GREEN} + T_{YELLOW}$.

Ba mode 2, 3, 4 dùng chung một đoạn code; hàm `colorOfMode()` cho biết mode hiện tại đang chỉnh màu nào, và `saveDuration()` lưu giá trị theo màu đó.

10 Exercise 10: Final project

Toàn bộ chương trình nằm trong `main.c`. Ngắt TIM2 chỉ đọc nút và đếm software timer; mọi xử lý nằm trong hàm `fsm_run()` được gọi liên tục trong `while(1)`.

Listing 5: Biến trạng thái và các hàm hỗ trợ của FSM

```
#define INIT        0
#define AUTO        1   /* Mode 1 */
#define MAN_RED     2   /* Mode 2 */
#define MAN_YELLOW  3   /* Mode 3 */
#define MAN_GREEN   4   /* Mode 4 */

int status = INIT;
int statusHor, statusVer;        /* mau hien tai cua 2 huong */
int countHor, countVer;          /* so giay con lai          */
int durRed = 5, durYellow = 2, durGreen = 3;   /* luon giu DO = XANH + VANG */
int tempValue = 0;               /* gia tri dang chinh o mode 2-4 */

int nextColor(int color) {       /* RED -> GREEN -> YELLOW -> RED */
    if (color == LIGHT_RED)   return LIGHT_GREEN;
    if (color == LIGHT_GREEN) return LIGHT_YELLOW;
    return LIGHT_RED;
}

int durationOf(int color) {
    if (color == LIGHT_RED)   return durRed;
    if (color == LIGHT_GREEN) return durGreen;
    return durYellow;
}

int colorOfMode(int mode) {
    if (mode == MAN_RED)    return LIGHT_RED;
    if (mode == MAN_YELLOW) return LIGHT_YELLOW;
    return LIGHT_GREEN;
}

/* Luu gia tri moi, giu DO = XANH + VANG de 2 huong khong cung xanh */
void saveDuration(int color, int value) {
    if (color == LIGHT_RED) {
        if (value < 2) value = 2;
        durRed = value;
        durGreen = durRed - durYellow;              /* sua XANH cho khop */
        if (durGreen < 1) { durGreen = 1; durYellow = durRed - 1; }
    } else {
        if (color == LIGHT_YELLOW) durYellow = value;
        else                       durGreen  = value;
        durRed = durGreen + durYellow;              /* sua DO cho khop */
        if (durRed > 99) {
            durRed = 99;
            if (color == LIGHT_YELLOW) durGreen  = 99 - durYellow;
            else                       durYellow = 99 - durGreen;
        }
    }
}

void enterAuto(void) {
    status = AUTO;
    statusHor = LIGHT_RED;    countHor = durRed;     /* Init (Hor) -> RED   */
    statusVer = LIGHT_GREEN;  countVer = durGreen;   /* Init (Ver) -> GREEN */
    setLightHor(statusHor);
    setLightVer(statusVer);
    setBufferA(countHor);
    setBufferB(countVer);
```

```
    setTimer(TMR_SECOND, 1000);
}

void enterManual(int mode) {
    status = mode;
    setLightHor(LIGHT_OFF);
    setLightVer(LIGHT_OFF);
    tempValue = durationOf(colorOfMode(mode));
    setBufferA(tempValue);       /* cap A: gia tri dang chinh  */
    setBufferB(mode);            /* cap B: so mode 02, 03, 04  */
    setTimer(TMR_BLINK, 250);    /* dao moi 250 ms -> nhay 2 Hz */
}

void increaseValue(void) {
    tempValue++;
    if (tempValue > 99) tempValue = 1;
    setBufferA(tempValue);
}
```

Listing 6: FSM chính

```
void fsm_run(void) {
    switch (status) {
    case INIT:
        setTimer(TMR_SCAN, 10);
        enterAuto();
        break;

    case AUTO:
        if (isTimerExpired(TMR_SECOND)) {
            setTimer(TMR_SECOND, 1000);
            countHor--;
            if (countHor <= 0) { statusHor = nextColor(statusHor); countHor =
    durationOf(statusHor); }
            countVer--;
            if (countVer <= 0) { statusVer = nextColor(statusVer); countVer =
    durationOf(statusVer); }
            setLightHor(statusHor);
            setLightVer(statusVer);
            setBufferA(countHor);
            setBufferB(countVer);
        }
        isButtonPressed(BTN_INC);            /* bo qua nut 2, 3 o mode 1 */
        isButtonPressed(BTN_SET);
        if (isButtonPressed(BTN_MODE)) enterManual(MAN_RED);
        break;

    case MAN_RED:
    case MAN_YELLOW:
    case MAN_GREEN:
        if (isTimerExpired(TMR_BLINK)) {
            setTimer(TMR_BLINK, 250);
            toggleColor(colorOfMode(status));
        }
        /* nut 2: nhan +1, giu qua 1 s tu +1 moi 200 ms */
        if (isButtonPressed(BTN_INC)) {
            increaseValue();
            setTimer(TMR_REPEAT, 200);
```

```
        }
        if (isButtonLongPressed(BTN_INC) && isTimerExpired(TMR_REPEAT)) {
            increaseValue();
            setTimer(TMR_REPEAT, 200);
        }
        /* nut 3: luu gia tri */
        if (isButtonPressed(BTN_SET)) {
            saveDuration(colorOfMode(status), tempValue);
            tempValue = durationOf(colorOfMode(status));
            setBufferA(tempValue);
        }
        /* nut 1: MAN_RED -> MAN_YELLOW -> MAN_GREEN -> Automatic */
        if (isButtonPressed(BTN_MODE)) {
            if (status == MAN_GREEN) enterAuto();
            else                     enterManual(status + 1);
        }
        break;
    }

    /* quet 7 doan moi 10 ms: 2 LED moi cap -> 50 Hz */
    if (isTimerExpired(TMR_SCAN)) {
        setTimer(TMR_SCAN, 10);
        scan7SEG();
    }
}
```

Listing 7: Hàm main

```
  /* USER CODE BEGIN 2 */
  HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin, GPIO_PIN_SET);  /* tat 7 doan */
  HAL_GPIO_WritePin(GPIOB, EN2_Pin | EN3_Pin, GPIO_PIN_SET);
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */
  while (1)
  {
    fsm_run();
  }
```

Kết quả

- Mode 1: hướng ngang đỏ 5 s → xanh 3 s → vàng 2 s, hướng dọc lệch pha; hai cặp LED 7 đoạn đếm ngược thời gian còn lại.
- Mode 2–4: đèn của màu đang chỉnh nháy 2 Hz, cặp A hiện giá trị, cặp B hiện số mode; B2 tăng, B3 lưu.
- Sau khi quay về Mode 1, hệ thống chạy với thời gian mới.